Checkout Incident Detection Agent

A two-agent n8n pipeline that triages checkout events against deterministic rules and a failure baseline, publishing exactly one ALERT-or-LOG command per run — with an LLM used only to explain the call, never to make it.

PROBLEM

E-commerce teams need to catch checkout issues — especially payment failures — at a volume too high to triage by hand. The brief called for an agent that reads new checkout events, judges severity against a normal baseline, and hands off exactly one clean command (ALERT/LOG) per run without executing fixes itself. Originally scoped for Zapier; built on n8n Cloud instead.

APPROACH & KEY DECISIONS

- Deterministic code decides severity/action (PAYMENT step, cart value threshold, baseline overrun, repeated mobile failures) — the LLM (Gemini, temperature 0) only writes the one-sentence *reason*, so the actual decision stays reproducible and testable.
- Triage runs on a 15-minute schedule, not per-event, to stay inside free-tier LLM rate limits while keeping the batch logic simple.
- Built test-first: five curl scenarios against n8n's Test URL, then a self-contained HTML test console (no server) once hand-editing JSON per run became the bottleneck.

WHAT IT DOES

- **Agent 1** — webhook ingestion: validates and normalizes each event, appends it to the Cart Events sheet, responds 200 or a field-specific 400.
- **Agent 2** — scheduled triage: reads only unprocessed rows (state-gated), evaluates each checkout step against the four HIGH-severity rules, gets a one-line reason from Gemini, writes one command row, advances state, and posts to Slack.

RESULTS & LEARNINGS

- Confirmed end-to-end on a live run: a SHIPPING/MOBILE event (cart value 135,672) correctly triggered ALERT/HIGH, matching across the sheet row, execution log, and Slack message.
- Testing with deliberately broken input (not just the happy path) surfaced real defects: a hard crash on non-numeric cart_value, a missing device-validation check, a generic 400 message, and a webhook response-mode misconfiguration that silently swallowed both success and error responses.
- Known gap: the spec calls a downstream webhook broadcast mandatory; this build hands off via a Slack post instead and does not include an outbound HTTP Request node.

TECH STACK

Component	Technology
Platform	n8n Cloud (app.n8n.cloud)
Data store	Google Sheets — 4 tabs (OAuth2)
Logic	n8n Code nodes (JavaScript) + Set/If
LLM	Google Gemini Chat Model, temp 0 (Google AI Studio key)
Notification	Slack Bot Token (chat:write, users:read*)
Test harness	Self-contained HTML/JS console, no server